Νευρωνικά Δίκτυα – Βαθιά Μάθηση : Εργασία 1

Δημήτριος Μουταφτσίδης 9920 (ΤΗΜΜΥ)

moutdimi@ece.auth.gr

Abstract.

Η εργασία υλοποιεί ένα Συνελικτικό Νευρωνικό Δίκτυο (CNN) για την ταξινόμηση της βάσης δεδομένων CIFAR-10. Υλοποιείται μέσω της βιβλιοθήκης PyTorch. Περιλαμβάνει ένα .ipynb αρχείο με κώδικα , σχόλια επί αυτού και τεχνικές λεπτομέρειες αναφορικά με την προγραμματιστική υλοποίηση. Το θεωρητικό κομμάτι και τα αποτελέσματα παρουσιάζονται στην παρούσα αναφορά. Θα δούμε πως τα configurations επηρεάζουν τα αποτελέσματα της ταξινόμησης σε χρόνο και ακρίβεια.

1.0. Data Augmentation.

Η τροποποίηση των δεδομένων είναι πολύ αποτελεσματικό εργαλείο για βελτίωση της επίδοσης. Ειδικά στην περίπτωση των Image Classifiers. Πόσο μάλλον των CNN. Η διαπίστωση αυτή είναι γνωστή τόσο από την θεωρία, αλλά και από προηγούμενη εμπειρία του συγγραφέα. Δεν αποδεικνύεται με πείραμα στην παρούσα εργασία. Είναι, άλλωστε best and common practice για τέτοιου τύπου assignments.

1.1. Η Γενική Ιδέα.

Προτού το δίκτυο «δει» τα δεδομένα για να εκπαιδευτεί, αυτά δειγματοληπτικά υφίστανται ορισμένες τροποποιήσεις. Αλλαγές στον χρωματισμό, στον προσανατολισμό, το μέγεθος κτλ. Αποσκοπεί στην βελτίωση της γενίκευσης και την αποφυγή overfitting. Εισάγει θόρυβο, στοχαστικότητα και «δυσκολίες» εκπαίδευσης. Το μοντέλο μαθαίνει να μην βασίζεται μόνο σε volatile features (όπως πχ η απόχρωση) για την ταξινόμηση. Έτσι γενικεύει καλύτερα και επιτυγχάνεται μεγαλύτερη ευρωστία. Δηλαδή ανοχή σε θόρυβο και σε test sets με ελαφρώς διαφορετικές κατανομές από τα training data. Χρειάζεται προσοχή σχετικά με το πόσο παρεμβατικοί γινόμαστε με τα Augmentations. Εξαρτάται από το μοντέλο και το data set. Βαθιά , μεγάλα μοντέλα με πολλές παραμέτρους επωφελούνται από πιο «βαριές» τροποποιήσεις. Το ίδιο ισχύει και για πολύ μονότονα datasets. Ρηχά μοντέλα, ή ήδη θορυβώδη δεδομένα μπορεί να οδηγήσουν σε underfitting. Στην υποενότητα 1.2 εξετάζεται κάθε μία από τις τεχνικές.

1.2. Υλοποιημένες Τεχνικές.

1.2.1 Random Horizontal Flip.

Τυχαία περιστρέφει την εικόνα ως προς τον οριζόντιο άξονα. Μαθαίνει στο μοντέλο να αντιλαμβάνεται καλύτερα την γεωμετρία των αντικειμένων. Σχετικά ήπια παρέμβαση.

1.2.2. Cropping (με Mirroring).

Εισάγει shift στην γεωμετρία της εικόνας. Προσαυξάνει την διάσταση με pixels παρόμοιας μορφής με αυτά της εικόνας (mirroring). Έπειτα επιστρέφει ένα τυχαίο στιγμιότυπο της προσαυξημένης, ίδιων διαστάσεων με την αρχική. Έτσι, το μοντέλο δεν βασίζεται τόσο στην τοποθέτηση του αντικειμένου ενδιαφέροντος πάνω στην εικόνα για την ταξινόμηση. Σχετικά ήπια τεχνική.

1.2.3. Color Jittering.

Μεταβάλει τυχαία την φωτεινότητα, τον κορεσμό και το contrast της εικόνας σε προκαθορισμένο διάστημα. Ωθεί το μοντέλο να εστιάζει και στο σχήμα, όχι μόνο στο color content. Πολύ αποτελεσματική με κατάλληλη επιλογή παραμέτρων. Μπορεί να είναι ήπια και πολύ παρεμβατική, ανάλογα με τα διαστήματα που ορίζουμε.

1.2.4. Random Erasing.

Τυχαία επιλέγει ένα ορθογώνιο τμήμα της εικόνας και το αντικαθιστά με θόρυβο. Η διαγραφή αυτή συμβαίνει με πιθανότητα και σε μέγεθος που επιλέγουμε εμείς. Μαθαίνει στο μοντέλο να μην βασίζεται σε πολύ συγκεκριμένες μορφολογίες για την ταξινόμηση. Είναι αρκετά παρεμβατική. Συνιστάται για μεγάλα μοντέλα.

1.2.5. Normalization.

Δεν αποτελεί Augmentation. Είναι προϋπόθεση για καλή λειτουργία. Τα δεδομένα πρέπει να έχουν (σχεδόν) μηδενική μέση τιμή. Τα Gradients υπολογίζονται ευκολότερα, οι Gradient Descent μέθοδοι συγκλίνουν γρηγορότερα, δεν φεύγουμε από την ενεργή περιοχή των συνηθισμένων activation functions κ.α. Για την κανονικοποίηση του Test Set χρησιμοποιούμε τα στατιστικά των Training Data. Αυτό είναι απαραίτητο για μία σωστή αξιολόγηση. Τα Test Data πρέπει να μας είναι ουσιαστικά αόρατα. Υποθέτουμε πως έχουν παρόμοια κατανομή με τα δεδομένα εκπαίδευσης.

2.0. Convolutional Neural Networks.

Χρησιμοποιούνται κατά κόρον σε Image Classification Tasks. Είναι ειδική περίπτωση NN. Ας δούμε αρχικά την βασική ιδέα που διέπει την λειτουργία τους, πως διαφέρουν από τα κλασσικά MLP και γιατί είναι πιο αποτελεσματικά στην περίπτωση μας.

2.1. Βασικές Αρχές Λειτουργίας.

Η διαδικασία μάθησης συνοψίζεται ως εξής:

1) Forward Pass.

Το δίκτυο κάνει μία πρόβλεψη με βάση το input. Στα πρώτα στάδια εκπαίδευσης είναι χαμηλής επίδοσης, αφού τα βάρη αρχικοποιούνται τυχαία.

2) Υπολογισμός Loss.

Χρησιμοποιείται συνάρτηση για την ποσοτικοποίηση του σφάλματος μεταξύ προβλεπόμενων και πραγματικών τιμών.

3) Backwards Pass – Backpropagation.

Υπολογισμός των Gradients του loss function συναρτήσει όλων των βαρών. Προσδιορίζεται έτσι η κατεύθυνση στην οποία πρέπει να «κινηθούν» τα βάρη ώστε να μειωθεί το σφάλμα.

4) Weight Update.

Ενημέρωση των βαρών για την επίτευξη του παραπάνω σκοπού.

Ας δούμε τώρα πως η παραπάνω διαδικασία υλοποιείται από ένα CNN.

2.2. CNN – Specifics.

Για να καταλάβουμε την διαδικασία θα δούμε ένα παράδειγμα για ένα sample και ένα training epoch. Ξεκινάμε με το forward pass. Έστω x , k , b το sample, η συνελικτική μάσκα και το bias αντίστοιχα. Έστω επίσης $\sigma(\ldots)$ κάποιο activation function. Αρχικά υλοποιείται η συνέλιξη με το φίλτρο: $z = k * x + b$. Στην συνέχεια εφαρμόζεται το activation function: $a = \sigma(z)$. Για να προχωρήσουμε στο επόμενο layer, θεωρούμε ως input το output του προηγούμενου. Επαναλαμβάνοντας για όλα τα layers, έχουμε ολοκληρώσει το πρώτο στάδιο. Σημειώνουμε πως μπορεί να ακολουθούν και διεργασίες όπως batch normalization, pooling κτλ. τα οποία θα δούμε αργότερα. Για το loss calculation, λαμβάνουμε υπόψιν τις προβλέψεις στο output layer. Στην περίπτωση του Cross Entropy Loss criterion που χρησιμοποιούμε στο μοντέλο, έχουμε: $Loss = -\sum_k y_k \log(\hat{y}_k)$. Το y_k είναι μεταβλητή δείκτης. Είναι 1 αν η κλάση είναι k και 0 αλλιώς. Ως $\hat{y}_k$ ορίζουμε την πιθανότητα που δίνει το μοντέλο στην υπό εξέταση κλάση. Δηλαδή με απλά λόγια: για κάθε κλάση, αθροίζουμε τον λογάριθμο της «σιγουριάς» του μοντέλου να είναι όντως αυτή η κλάση. Αν το μοντέλο απαντά με 100% βεβαιότητα, η εν λόγω ταξινόμηση δεν προσθέτει loss (αφού $\log(1) = 0$). Μικρή assigned probability προσθέτει σημαντικά. Προχωράμε πλέον στο back propagation step. Υπολογίζονται τρία Gradients: $\delta_z = \delta\sigma'(z)$, $\frac{\partial L}{\partial k} = \delta_z x$, $\frac{\partial L}{\partial b} = \delta_z$, $\frac{\partial L}{\partial x} = \delta_z k$. Το δ_z για το εκάστοτε layer έχει υπολογιστεί όπως φαίνεται στην πρώτη ισότητα, ξεκινώντας από το Output Layer και ορίζοντας αρχικά $\delta_{out} = \hat{y} - y$. Με απλά λόγια: ξεκινώντας από το τέλος, υπολογίζουμε το πόσο το loss επηρεάζεται από τα βάρη των φίλτρων, το bias και το input. Καταλαβαίνουμε λοιπόν πως πρέπει να ενημερωθούν τα βάρη ώστε να «κινηθούμε στην κατεύθυνση» της μεγαλύτερης ακρίβειας. Τέλος, τα weight updates γίνονται από τον Optimizer. Θα δούμε αργότερα ποιόν χρησιμοποιούμε και γιατί.

Συμπέρασμα: Τα CNN δεν «βλέπουν» την εικόνα. Βλέπουν την συνέλιξη αυτής με κάποια φίλτρα (μάσκες). Αυτές ακριβώς είναι και οι παράμετροι πάνω στις οποίες εκπαιδεύουμε. Σκοπός είναι η εξαγωγή των κατάλληλων μασκών, ώστε να οδηγηθούμε στα επιθυμητά αποτελέσματα.

2.3. Πλεονεκτήματα CNN για Image Classification.

Τα CNN αποδίδουν πολύ καλά για τρεις βασικούς λόγους. Πρώτον, αντιλαμβάνονται την χωρική δομή των εικόνων. Ένα MLP χειρίζεται την εικόνα σαν διάνυσμα από pixel values. Έννοιες όπως η χωρική συχνότητα και μορφολογίες όπως ακμές και σχήματα χάνονται τελείως. Με την χρήση φίλτρων, το μοντέλο εκπαιδεύεται στα χωρικά μοτίβα που συνθέτουν μια εικόνα. Δεύτερον, είναι εξαιρετικά φειδωλά όσον αφορά την χρήση παραμέτρων. Η κάθε μάσκα χρησιμοποιείται για ολόκληρη την εικόνα. Για RGB διαστάσεων 32x32, χρειαζόμαστε μόνο 3x3x3 = 27 βάρη. Στην περίπτωση του MLP θα θέλαμε 32x32x3 = 3072! Τρίτον, αδιαφορούν για την τοποθέτηση μίας μορφής στον χώρο. Μια (ίδια) γάτα στο πάνω δεξί μέρος της εικόνας δίνει την ίδια συνέλιξη με μία στο κάτω αριστερά. Είναι ανθεκτικά σε Shifts. Ένα MLP θα χειριζόταν αυτή την αλλαγή σαν άσχετο sample.

Εν συντομία: τα CNN αξιοποιούν την χωρική δομή της εικόνας, το οποίο είναι ίσως το πιο σημαντικό χαρακτηριστικό για την ταξινόμηση.

3.0. Το Μοντέλο μας – Kasper.

Ας εξετάσουμε την αρχιτεκτονική που χρησιμοποιήθηκε για μοντέλο. Αλλάζει ελαφρώς για κάθε πείραμα. Στην παρούσα ενότητα θα δούμε τις παραμέτρους που κρατήθηκαν σταθερές. Επηρεάζουν και αυτές την επίδοση του μοντέλου και είναι παράγοντας που αξίζει διερεύνηση. Δεδομένου όμως του μεγάλου χρόνου εκπαίδευσης, τα πειράματα επικεντρώνονται στο βάθος και το πλάτος του δικτύου και στον τρόπο επαύξησης των δεδομένων.

3.1. Σταθερά Χαρακτηριστικά.

3.1.1. Activation Function.

Χρησιμοποιήθηκε ReLU. Θα λέγαμε πως είναι το default choice για τα περισσότερα NN. Ορίζεται ως $ReLU(x) = \max(0, x)$. Κοινώς, x if x > 0 else 0. Είναι πολύ «φθηνή» υπολογιστικά. Είναι αξιόπιστη και χρησιμοποιείται πολύ στην βιβλιογραφία για tasks σαν το δικό μας. Για εύρωστη λειτουργία συνδυάζεται με Batch Normalization.

3.1.2. Batch Normalization.

Επίσης κοινότυπη τεχνική που αναφέρεται αρκετά συχνά στην βιβλιογραφία. Μαθηματικά εκφράζεται ως: $\hat{x} =$

$\frac{x-\mu}{\sigma}$, $y = \gamma\hat{x} + \beta$. Τα γ , β είναι εκπαιδεύσιμα. Το B.N. κάνει compensate για το Internal Covariance Shift. Δηλαδή εξουδετερώνει την αλλαγή της κατανομής των layer outputs. Αυτό δύναται να αποβεί μοιραίο για την ReLU. Αν έχουμε shift προς τα αρνητικά, θα έχουμε συστηματικά μηδενικά outputs. Έγινε μικρό πείραμα χωρίς B.N. και η απόδοση ήταν περίπου 8% κάτω!

3.1.3. Max Pooling.

Χρησιμοποιείται μετά από κάθε Convolutional Layer με Kernel μεγέθους 2x2. Συμπυκνώνει ουσιαστικά τα outputs μειώνοντας τις διαστάσεις και διατηρώντας το πιο «δυνατό» activation μεταξύ των τεσσάρων στοιχείων. Καταλήγει σε υποδιπλασιασμό της κάθε διάστασης. Ας δούμε το εξής παράδειγμα: έστω channels = [32 , 64 , 128].

Input: 3x32x32 --- Conv>>>32x32x32 --- MaxPool 2x2>>> 32x16x16 --- Conv>>>64x16x16---MaxPool 2x2>>> 64x8x8. Γίνεται φανερό πως για 32x32 εικόνες μπορούμε να επαναλάβουμε την διαδικασία μέχρι 5 φορές. Για το μοντέλο Kasper επιλέξαμε να μην έχουμε Max Pooling στο τελευταίο Convolutional Block. Η επανάληψη μετά από κάθε Block ήδη είναι αρκετά δραστική. Η χρησιμότητα της παραπάνω διαδικασίας έγκειται στην μείωση του υπολογιστικού κόστους. Επίσης, λόγω διατήρησης των πιο ισχυρών activations, φιλτράρεται ως έναν βαθμό ο θόρυβος.

3.1.4. Convolution Kernel Size.

Χρησιμοποιούμε συνελικτικές μάσκες μεγέθους 3x3. Είναι πολύ κοινότυπη επιλογή. Είναι ο μικρότερος δυνατός πυρήνας που μπορεί να ανιχνεύσει χωρικές μεταβολές. Μεγαλύτερο μέγεθος, ακόμα και 5x5 θα επιβάρυνε πολύ την διαδικασία.

3.1.5. Scheduler.

Επιλέχθηκε StepLR (και Multistep, αλλά είναι πολύ παρόμοιας φιλοσοφίας). Μετά από δοσμένο αριθμό epochs, επιβραδύνει τον ρυθμό μάθησης. Έτσι, στην αρχή έχουμε γρήγορη αύξηση επίδοσης, ενώ αργότερα εξασφαλίζουμε καλύτερη και πιο ισορροπημένη σύγκλιση. Χωρίς Scheduler, δύναται το μοντέλο να ταλαντώνεται ή να εγκλωβιστεί γύρω από μη βέλτιστη λύση.

3.1.6. Optimizer.

Πρόκειται για τον αλγόριθμο ενημέρωσης των βαρών. Καλείται σε κάθε mini batch, μετά το backpropagation. Επιλέχθηκε ο Adam. Είναι πολύ καλός για την περίπτωση μας. Ενημερώνει τα βάρη πιο επιθετικά σε κατευθύνσεις με σταθερά gradients και πιο συγκρατημένα σε κατευθύνσεις χαοτικών αλλαγών. Είναι ,δηλαδή, πιο προσαρμοστικός στα δεδομένα από το κλασσικό Stochastic Gradient Descent.

3.1.7. Dropout.

Επιλέξαμε την τιμή 0.5 (στα μεγάλης κλίμακας πειράματα). Δηλαδή σε κάθε training pass, το 50% των νευρώνων του κάθε layer απενεργοποιείται. Έτσι το NN αναγκάζεται να κατανέμει την πληροφορία σε περισσότερους, ενισχύοντας έτσι την ευρωστία. Αποφεύγεται το overfitting και η γενίκευση βελτιώνεται. Μεγαλύτερα δίκτυα επωφελούνται από μεγαλύτερο dropout. Όσο περισσότερες παραμέτρους έχουμε, τόσο μεγαλύτερος ο κίνδυνος στεγνής απομνημόνευσης των training data.

3.1.8. Hidden Fully Connected Layer.

Το output layer από 10 μόλις νευρώνες, αφού έχουμε δέκα ακριβώς κλάσεις. Μετά το πέρας του Feature Extractor, τα features που έχουν εξαχθεί απαριθμούνται κατά αρκετές εκατοντάδες, ίσως και χιλιάδες. Το διάνυσμα που προκύπτει είναι πολύ μεγάλων διαστάσεων. Αν τρέχαμε κατευθείαν το Linear Classifier Head, η ταξινόμηση θα ήταν αργή και ίσως λιγότερο αποτελεσματική. Έτσι, γίνεται χαρτογράφηση σε διάνυσμα διάστασης 256. Αυτό αφορά τα πειράματα για τα ικανά μοντέλα. Θα συμπεριληφθούν δύο πειράματα στο τέλος σε μοντέλα μικρής κλίμακας για να ικανοποιηθεί το ζητούμενο της εκφώνησης.

3.1.9. Mini Batch Size.

Επιλέχθηκαν 128 δείγματα ανά batch. Είναι τιμή που φαίνεται να ισορροπεί ταχύτητα και επίδοση. Μικρότερα batches βελτιώνουν ακρίβεια και επιβαρύνουν τον χρόνο εκπαίδευσης. Το αντίστροφο για μεγαλύτερα.

Ας δούμε τώρα τα χαρακτηριστικά του δικτύου που τροποποιούμε για τις ανάγκες των πειραμάτων.

3.2. Μεταβλητά Χαρακτηριστικά.

3.2.1. Depth.

Το μήκος του διανύσματος channel επηρεάζει τον αριθμό των layers. Κάθε επιπλέον στοιχείο προσθέτει ένα νέο συνελικτικό block. Η ακριβής μέτρηση ποικίλει ανάλογα με το τι ο καθένας θεωρεί ως layer. Οπότε ας συζητήσουμε καλύτερα σε επίπεδο pipelines. Κάθε νέο σετ νευρώνων στο channels δέχεται τις εξής μετατροπές: Convolution >> Batch Normalization >> ReLU >> (πλην του τελευταίου) Max Pooling. Έχουμε ήδη περιγράψει την διαδικασία αναλυτικά. Καταλαβαίνουμε πως περισσότερα channels ισοδυναμούν με μεγαλύτερο βάθος και ,συνήθως, πιο ικανό δίκτυο. Το classifier head έχει πάγια δύο layers.

3.2.2. Width.

Αφορά τον αριθμό των νευρώνων που επιλέγουμε για κάθε layer. Πλατύτερα δίκτυα τείνουν να είναι πιο αργά. Είναι όμως ,συνήθως, πιο αποδοτικά στην ταξινόμηση. Από το πειραματισμό οδηγηθήκαμε στο συμπέρασμα πως το augmentation και το dropout για πολύ πλατιά δίκτυο είναι καλό να είναι πιο ενισχυμένο. Αποφεύγεται το overfitting.

3.2.3. Learning Rate.

Υποδεικνύει στον Optimizer το πόσο δραστικά να επέμβει στα βάρη. Μεγάλο LR επιταχύνει την σύγκλιση. Ενέχει

όμως κίνδυνο απόκλισης από το βέλτιστο. Αντίστοιχα, μικρό LR συνεπάγεται αργή, αλλά σταθερή σύγκλιση. Κινδυνεύει όμως με εγκλωβισμό σε κάποιο υποβέλτιστο plateau. Η χρήση κατάλληλου Scheduler μας επιτρέπει να αποκομίσουμε τα θετικά και των δύο «κόσμων».

3.2.4. Weight Decay.

Ενημερώνει τον Optimizer για το πόσο δραστικά να αποφύγει μεγάλα βάρη. Αποφεύγεται έτσι το overfitting. Η συστηματική συγκέντρωση μεγάλων βαρών υποδεικνύει απομνημόνευση των δεδομένων.

3.2.5. Augmentation Intensity.

Ο βαθμός τροποποίησης των δεδομένων επηρεάζει σημαντικά την επίδοση του δικτύου. Μεγαλύτερα δίκτυα επωφελούνται από «βαρύτερες» επαυξήσεις.

3.2.6. Epochs.

Αφορά το πόσες φορές «τρέχει» η εκπαίδευση του δικτύου. Επειδή στον κώδικα αποθηκεύουμε τον συνδυασμό βαρών με την καλύτερη επίδοση, δεν είναι τόσο μεγάλης σημασίας. Ωστόσο καλό είναι να επιλέγουμε αριθμό τέτοιο που και να επιτρέπει το μοντέλο να εξαντλήσει το potential που έχει, χωρίς όμως να οδηγεί σε παράλογα μεγάλους χρόνους εκπαίδευσης. Αν το μοντέλο φτάσει σε κορεσμό, τα εναπομείναντα epochs θα είναι περιττά.

Προτού προχωρήσουμε στα αποτελέσματα, καλό είναι να αναφέρουμε το εξής: <u>οι αλλαγές στις διάφορες παραμέτρους δεν είναι αμιγώς θετικές ή αρνητικές.</u> Πράγματι, για παράδειγμα, τα βαθύτερα δίκτυα τείνουν να λειτουργούν καλύτερα. Αυτό όμως προϋποθέτει την επιλογή σωστού augmentation, scheduling κτλ. <u>Οι διάφορες αρχιτεκτονικές πρέπει να λειτουργούν συνδυαστικά και σε αρμονία μεταξύ τους.</u> Ένα πολύ μεγάλο μοντέλο με πολύ ήπιο augmentation είναι πολύ πιθανό να είναι overfitted. Χρειάζεται λοιπόν να επιλέξουμε σωστό συνδυασμό. χαρακτηριστικών.

4.0. Πειράματα – Αποτελέσματα.

4.1. Το καλύτερο δίκτυο.

<u>Configuration</u>:

Βαρύ augmentation , channels: [64 , 128 , 256 , 256 , 512 , 512] , για 60 epochs με LR 10^{-3} , WD 10^{-4} και Step στα 30 epochs.

(όπως φαίνεται στο επισυναπτόμενο kasper_config_4_1.txt.)

<u>Χρόνος Εκπαίδευσης:</u>

160 λεπτά. 2,5 λεπτά / epoch

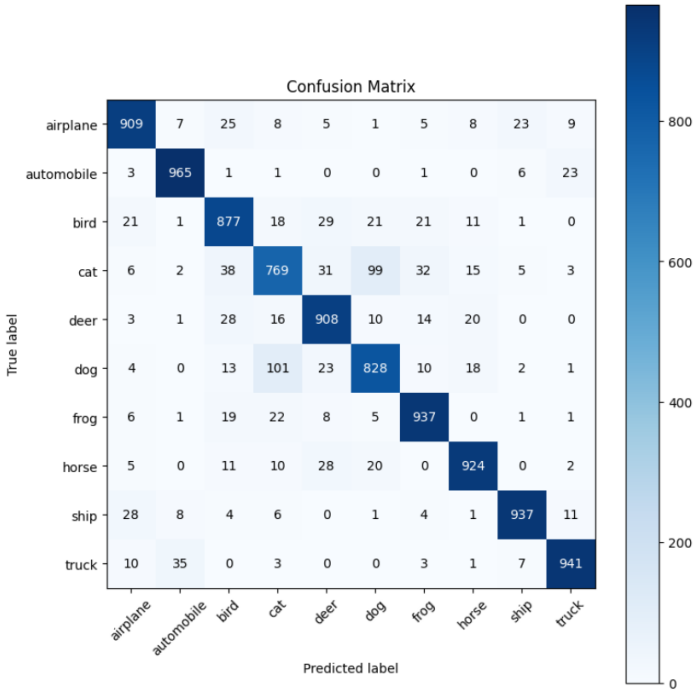
<u>Μέγιστη Επίδοση:</u>

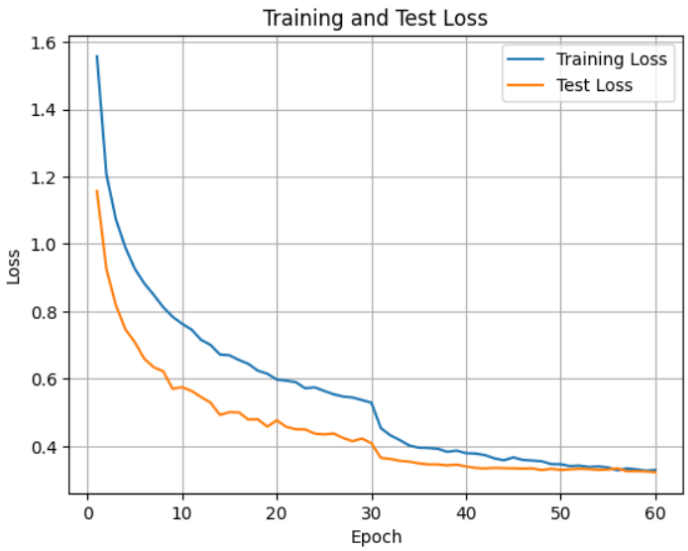
89.95% (δεν χρησιμοποιήθηκε seed, οπότε ποικίλει ελαφρώς ανάλογα με το training run-αυτό ισχύει για όλα τα πειράματα)

<u>Model stats:</u>

Classification Report:

	precision	recall	f1-score	support
airplane	0.91	0.91	0.91	1000
automobile	0.95	0.96	0.96	1000
bird	0.86	0.88	0.87	1000
cat	0.81	0.77	0.79	1000
deer	0.88	0.91	0.89	1000
dog	0.84	0.83	0.83	1000
frog	0.91	0.94	0.92	1000
horse	0.93	0.92	0.92	1000
ship	0.95	0.94	0.95	1000
truck	0.95	0.94	0.95	1000
accuracy			0.90	10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000



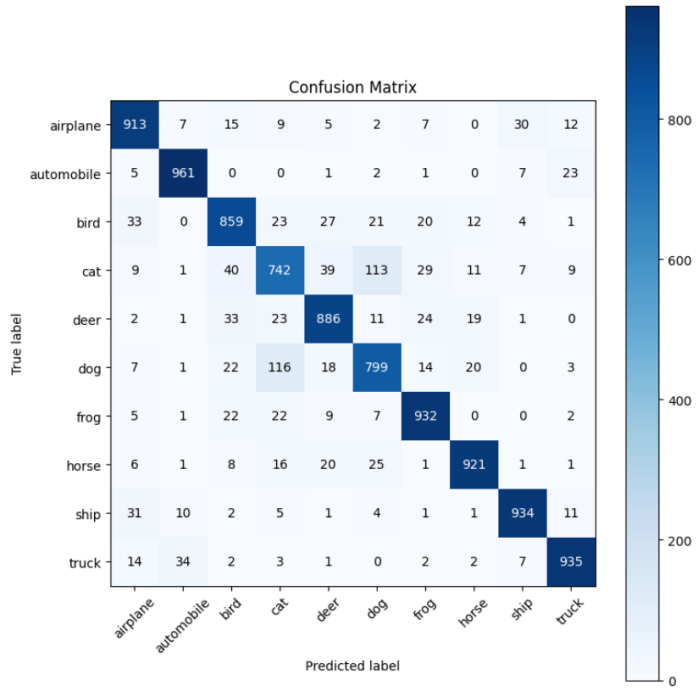


Classification Report:

	precision	recall	f1-score	support
airplane	0.89	0.91	0.90	1000
automobile	0.94	0.96	0.95	1000
bird	0.86	0.86	0.86	1000
cat	0.77	0.74	0.76	1000
deer	0.88	0.89	0.88	1000
dog	0.81	0.80	0.81	1000
frog	0.90	0.93	0.92	1000
horse	0.93	0.92	0.93	1000
ship	0.94	0.93	0.94	1000
truck	0.94	0.94	0.94	1000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Συμπεράσματα:

Πολύ ικανοποιητική επίδοση σε όλους τους τομείς. Καθαρό Confusion Matrix. Τα πιο πολλά «μπερδέματα» συμβαίνουν μεταξύ κλάσεων Cat και Dog. Λογικό, αφού είναι οι δύο πιο αδύναμες κλάσεις. Οι πιο «ισχυρές» είναι οι Automobile , Ship , Truck με F1 score 95%. Τα Training και Loss curves είναι πολύ «υγιή». Παρατηρούμε σταθερή σύγκλιση, με την επίδοση στα Test Data να είναι σταθερά πιο υψηλή. Αναμενόμενο λόγω των augmentations που καθιστούν την εκπαίδευση πιο δύσκολη. Αυτό υποδηλώνει πολύ καλή γενίκευση. Αποφύγαμε το overfitting. Η σύγκλιση επίσης φαίνεται επαρκής. Περισσότερα epochs δεν θα ωφελούσαν κατά πάσα πιθανότητα. Παρατηρούμε επίσης πως οι καμπύλες ομαλοποιούνται αισθητά μετά την ενεργοποίηση του Scheduler. Τα βάρη δίνονται στο αρχείο kasper_best_2.pth και το training performance στο kasper_performance.pth.



4.2. Πλατύτερη Εκδοχή του προηγούμενου δικτύου.

<u>Configuration:</u>

Ελαφρώς βαρύτερο augmentation, channels: [128 , 128 , 256 , 512 , 512 , 512] , για 140 epochs , με LR 10^{-4}, WD $5 \cdot 10^{-4}$, Multistep με milestones [50 , 100].

(όπως φαίνεται στο kasper_config_4_2.txt)

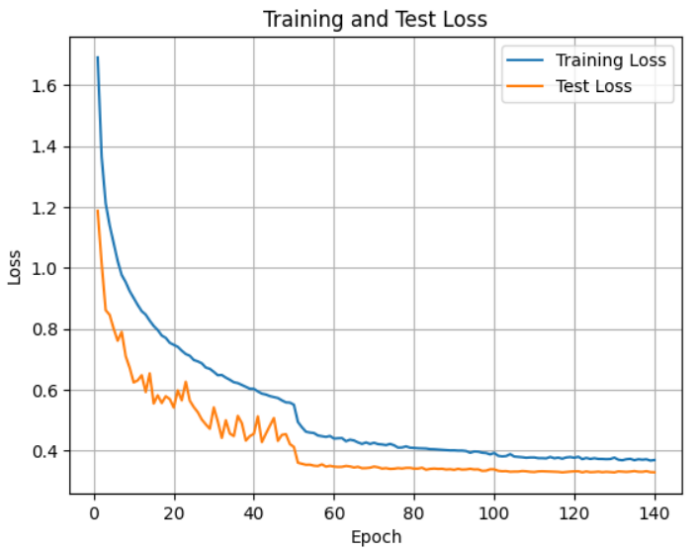
<u>Χρόνος Εκπαίδευσης:</u>

480 λεπτά. 3,5 λεπτά/epoch.

<u>Μέγιστη Επίδοση:</u>

88.82%

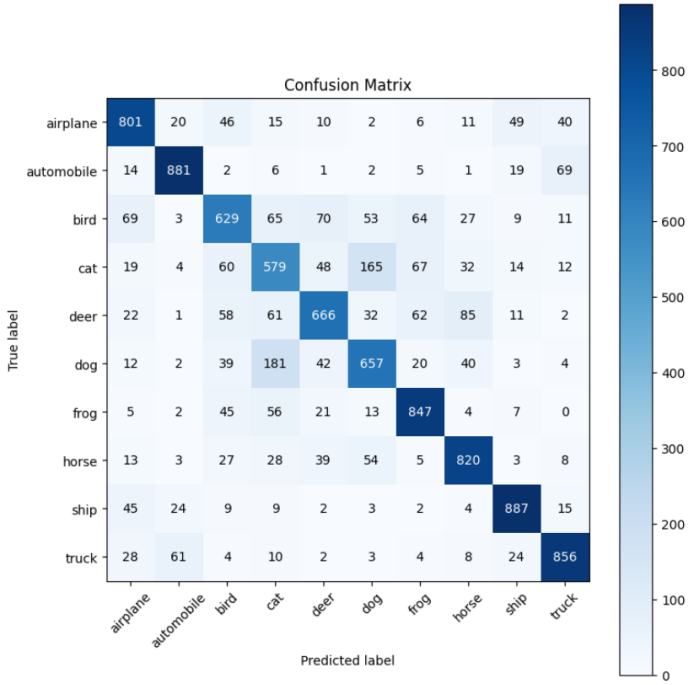
<u>Model Stats:</u>



```
Classification Report:

              precision    recall  f1-score   support

    airplane       0.78      0.80      0.79      1000
  automobile       0.88      0.88      0.88      1000
        bird       0.68      0.63      0.66      1000
         cat       0.57      0.58      0.58      1000
        deer       0.74      0.67      0.70      1000
         dog       0.67      0.66      0.66      1000
        frog       0.78      0.85      0.81      1000
       horse       0.79      0.82      0.81      1000
        ship       0.86      0.89      0.88      1000
       truck       0.84      0.86      0.85      1000

    accuracy                           0.76     10000
   macro avg       0.76      0.76      0.76     10000
weighted avg       0.76      0.76      0.76     10000
```

Συμπεράσματα:

Σκοπός ήταν η διερεύνηση της επιρροής του πλάτους στην επίδοση του NN ίδιου βάθους. Το augmentation επιλέχθηκε να είναι λίγο πιο ισχυρό. Το learning rate μικρότερο για πιο ομαλή σύγκλιση. Το πείραμα ήταν αποτυχία. Πήρε αρχικά 8 ώρες για να ολοκληρωθεί. Και το per epoch training time ήταν μεγαλύτερο. Αναμενόμενο, αφού τα layers έχουν περισσότερους νευρώνες. Η επίδοση κατέληξε να είναι σχεδόν ίδια. Η ιεραρχία κλάσεων διατηρείται επακριβώς. Αυτή είναι διαπίστωση που είχε παρατηρηθεί και στην ενδιάμεση εργασία μεταξύ 1-NN και 3-NN. Οι καμπύλες δεν φαίνεται να υποδεικνύουν overfitting. Στα test data παρατηρούνται ταλαντώσεις μέχρι την ενεργοποίηση του scheduler. Εκεί ομαλοποιείται, αλλά το μοντέλο συγκλίνει αρκετά αργά. Το δεύτερο Step είναι αχρείαστο. Ίσως καλύτερη επιλογή LR και Scheduler Gamma να έδινε καλύτερα αποτελέσματα. Πιο πολύ νευρώνες δεν συνεπάγονται απαραίτητα καλύτερη επίδοση. Τα βάρη δίνονται στο αρχείο kasper_best_strong.pth και το training performance στο kasper_performance_strong.pth.



4.3.1. Μικρό Μοντέλο 1.

<u>Configuration:</u>

Ελαφρύ augmentation, channels: [32 , 32] , για 20 epochs , με LR 10^{-3}, WD 10^{-3},Scheduler step στα 10 epochs.

(όπως φαίνεται στο kasper_config_4_3_1.txt)

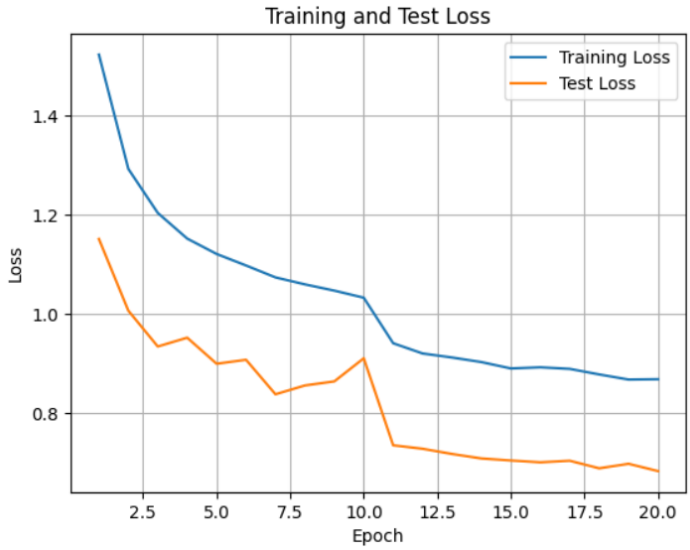
<u>Χρόνος Εκπαίδευσης:</u>

26 λεπτά. 1,3 λεπτά / epoch.

<u>Μέγιστη Επίδοση:</u>

76,3%

<u>Model Stats:</u>



Συμπεράσματα:

Πολύ ελαφρύ, αλλά σχετικά underfitted μοντέλο. Οι καμπύλες δεν συγκλίνουν επαρκώς. Ο σκοπός, ωστόσο του πειράματος είναι να δούμε πως θα επηρεαστεί η επίδοση με βάση αλλαγή μόνο στο βάθος του μοντέλου. Σημειώνουμε πως η ιεραρχία κλάσεων πάλι διατηρείται. Τα βάρη δίνονται στο αρχείο kasper_best_fast.pth και το training performance στο kasper_performance_fast.pth.

4.3.2. Μικρό Μοντέλο 2.

Configuration:

Ελαφρύ augmentation, channels: [32 , 32 , 32] , για 20 epochs , με LR 10^{-3}, WD 10^{-3},Scheduler step στα 10 epochs.

(όπως φαίνεται στο kasper_config_4_3_2.txt)

Χρόνος Εκπαίδευσης:

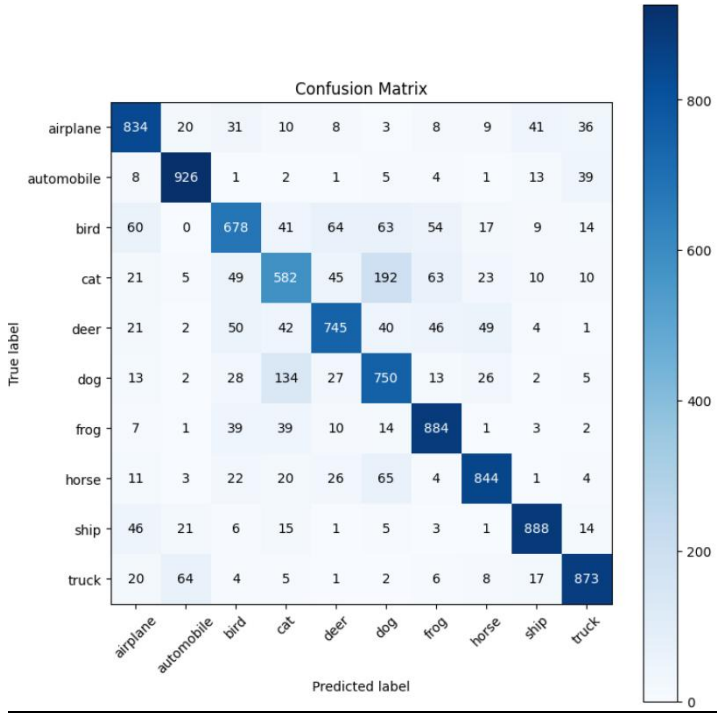
25 λεπτά. 1,25 λεπτά / epoch.

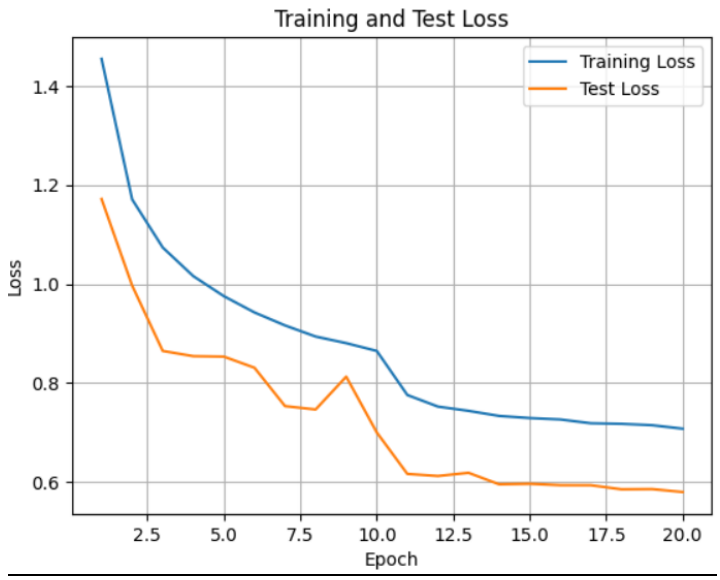
Μέγιστη Επίδοση:

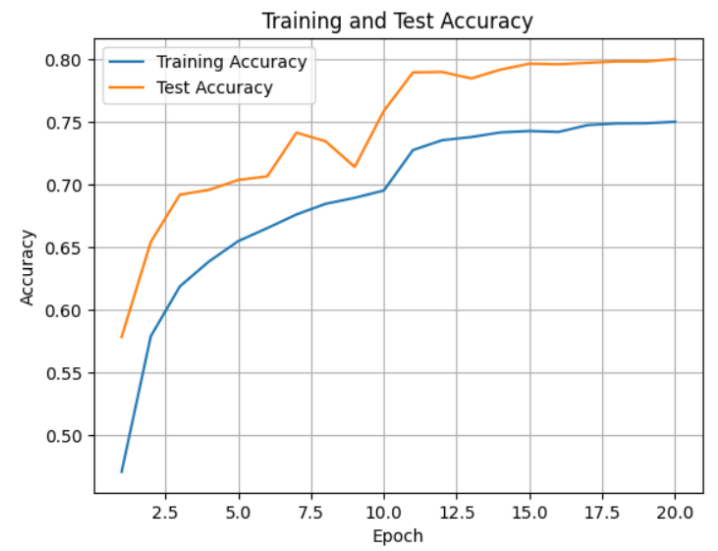
80%

Model Stats:

```
Classification Report:

              precision    recall  f1-score   support

    airplane       0.80      0.83      0.82      1000
  automobile       0.89      0.93      0.91      1000
        bird       0.75      0.68      0.71      1000
         cat       0.65      0.58      0.62      1000
        deer       0.80      0.74      0.77      1000
         dog       0.66      0.75      0.70      1000
        frog       0.81      0.88      0.85      1000
       horse       0.86      0.84      0.85      1000
        ship       0.90      0.89      0.89      1000
       truck       0.87      0.87      0.87      1000

    accuracy                           0.80     10000
   macro avg       0.80      0.80      0.80     10000
weighted avg       0.80      0.80      0.80     10000
```







Συμπεράσματα:

Η επίδοση είναι καλύτερη, όπως και αναμέναμε. Το παράδοξο είναι πως το runtime είναι το ίδιο. Μάλιστα, λίγο μικρότερο. Μπορέσαμε να σκεφτούμε δύο εξηγήσεις. Η πρώτη αφορά το PC. Τα μοντέλα τρέχουν στην CPU χωρίς

κάποιο code optimization όπως CUDA. Είναι πιθανό κάποια εσωτερική διεργασία των Windows που έτυχε να διενεργείται εκείνη την στιγμή να αύξησε κάπως τον χρόνο στο πείραμα 4.3.1. Ωστόσο το ξανατρέξαμε, και είχε μικρή χρονική απόκλιση από το πρώτο run. Η δεύτερη και πιο πιθανή εξήγηση αφορά το Max Pooling. Η προσθήκη ενός παραπάνω layer προσθέτει και έναν ακόμα κύκλο dimension reduction. Έτσι, στο fully connected layer έχουμε λιγότερες πράξεις πινάκων. Απ' ότι φαίνεται, η «οικονομία» σε εκείνο το στάδιο επισκιάζει τα κόστη των επιπλέον συνελίξεων. Τα βάρη δίνονται στο αρχείο kasper_best_fast_2.pth και το training performance στο kasper_performance_fast_2.pth.

Ας δούμε τώρα πως ο αριθμός των νευρώνων στο Fully Connected Layer επηρεάζει την απόδοση. Θα συγκρίνουμε με βάση το μοντέλο του πειράματος 4.3.2.

4.4.1. Μοντέλο με 512 Νευρώνες στο HL.

Configuration:

Ελαφρύ augmentation, channels: [32 , 32 , 32] , για 20 epochs , με LR 10^{-3}, WD 10^{-3},Scheduler step στα 10 epochs. Ο αριθμός των νευρώνων ορίστηκε ως 512. Χρειάζεται τροποποίηση στο αντίστοιχο line της κλάσης Kasper.

(όπως φαίνεται στο kasper_config_4_3_2.txt)

Χρόνος Εκπαίδευσης:

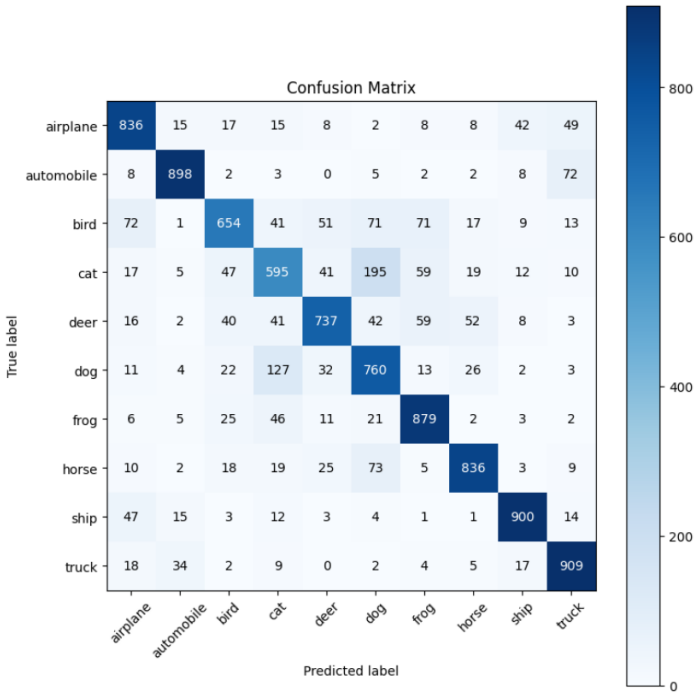
25 λεπτά. 1,5 λεπτά / epoch.

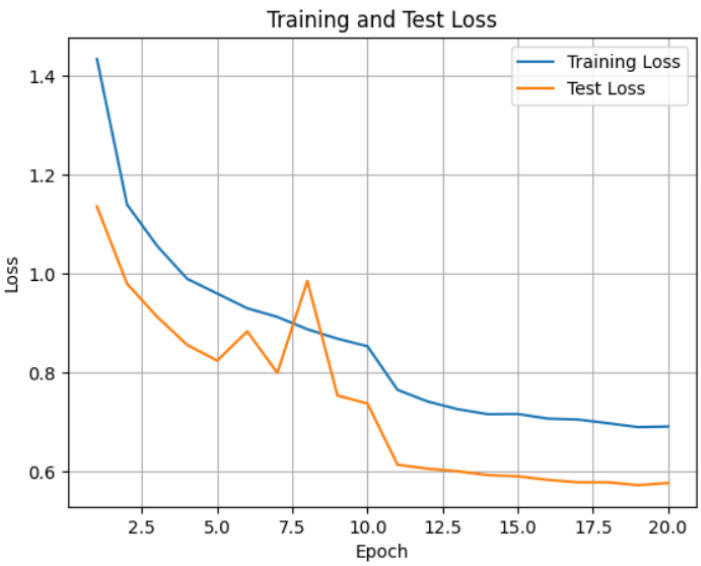
Μέγιστη Επίδοση:

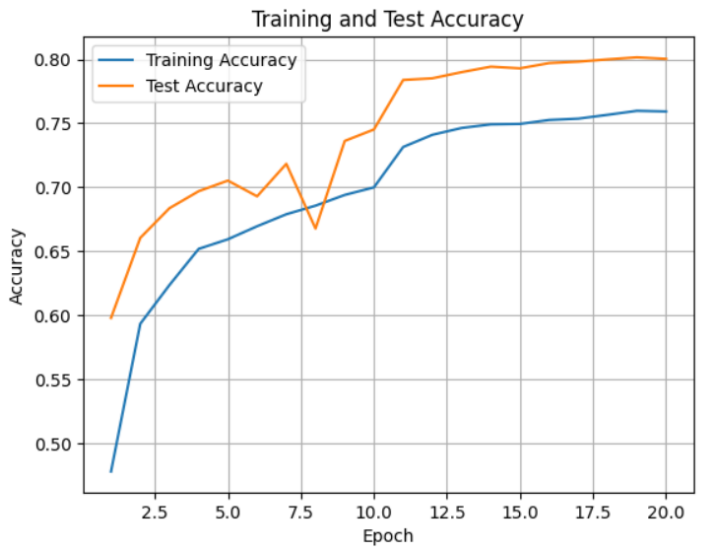
80%

Model Stats:

```
Classification Report:

              precision    recall  f1-score   support

    airplane       0.80      0.84      0.82      1000
  automobile       0.92      0.90      0.91      1000
        bird       0.79      0.65      0.71      1000
         cat       0.66      0.59      0.62      1000
        deer       0.81      0.74      0.77      1000
         dog       0.65      0.76      0.70      1000
        frog       0.80      0.88      0.84      1000
       horse       0.86      0.84      0.85      1000
        ship       0.90      0.90      0.90      1000
       truck       0.84      0.91      0.87      1000

    accuracy                           0.80     10000
   macro avg       0.80      0.80      0.80     10000
weighted avg       0.80      0.80      0.80     10000
```







Συμπεράσματα:

Τα δύο μοντέλα είναι σχεδόν ισοδύναμα. Αν εξαιρέσουμε την ιδιαιτερότητα στα training curves, όπου τα test και loss accuracies για ένα epoch είναι χειρότερα από αυτά του training. Είναι η μόνη διαφορά. Θεωρητικά έχει μεγαλύτερο

learning capacity. Τα epochs φαίνεται να ήταν λίγα για να εξάγουμε συμπέρασμα. Αν και υπάρχει ένα key takeaway από αυτό το πείραμα το οποίο θα σχολιαστεί στο τέλος. Τα βάρη δίνονται στο αρχείο kasper_best_512.pth και το training performance στο kasper_performance_512.pth.

Ας προχωρήσουμε σε πείραμα για 128 layers με το ίδιο setup.

4.4.2. Μοντέλο με 128 Νευρώνες στο HL.

Configuration:

Ελαφρύ augmentation, channels: [32 , 32 , 32] , για 20 epochs , με LR 10^{-3}, WD 10^{-3},Scheduler step στα 10 epochs. Ο αριθμός των νευρώνων ορίστηκε ως 128. Χρειάζεται τροποποίηση στο αντίστοιχο line της κλάσης Kasper.

(όπως φαίνεται στο kasper_config_4_3_2.txt)

Χρόνος Εκπαίδευσης:

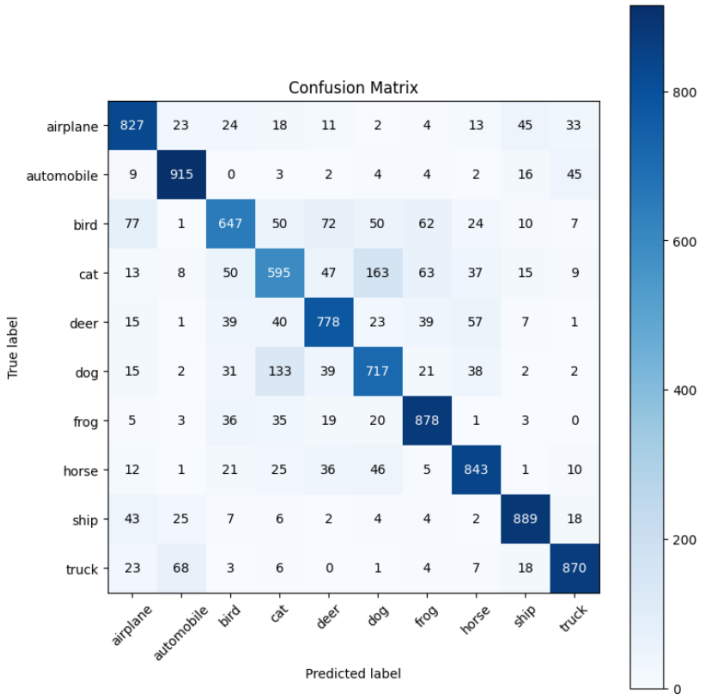
25 λεπτά. 1,5 λεπτά / epoch.

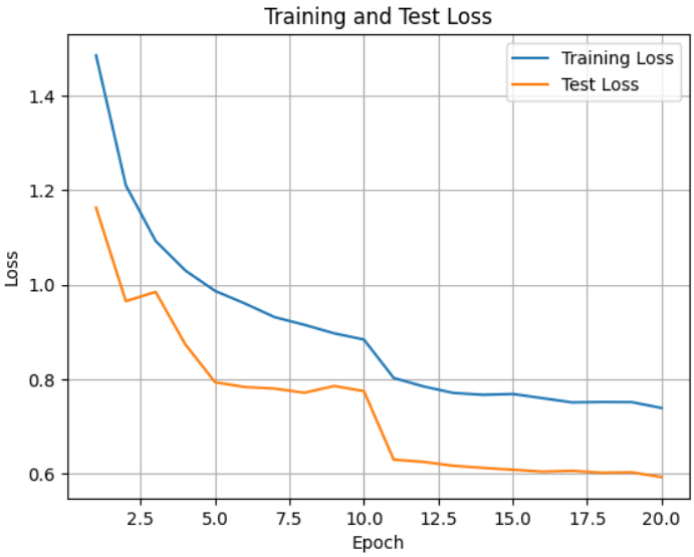
Μέγιστη Επίδοση:

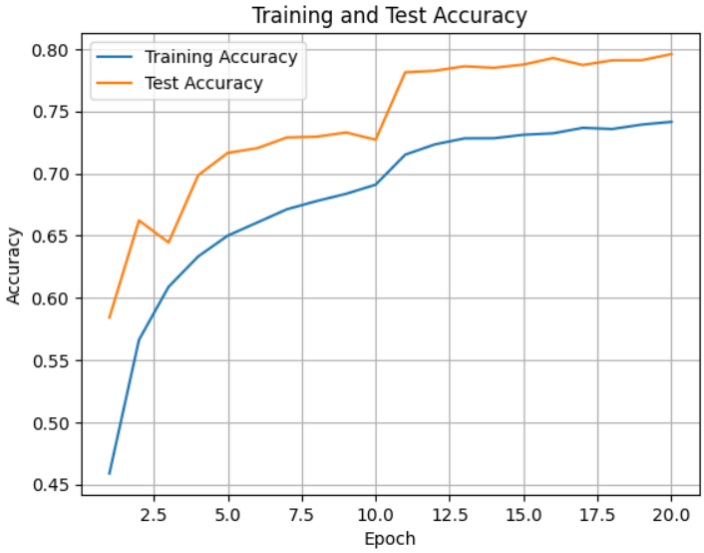
79%

Model Stats:

```
Classification Report:

              precision    recall  f1-score   support

    airplane       0.80      0.83      0.81      1000
  automobile       0.87      0.92      0.89      1000
        bird       0.75      0.65      0.70      1000
         cat       0.65      0.59      0.62      1000
        deer       0.77      0.78      0.78      1000
         dog       0.70      0.72      0.71      1000
        frog       0.81      0.88      0.84      1000
       horse       0.82      0.84      0.83      1000
        ship       0.88      0.89      0.89      1000
       truck       0.87      0.87      0.87      1000

    accuracy                           0.80     10000
   macro avg       0.79      0.80      0.79     10000
weighted avg       0.79      0.80      0.79     10000
```







Συμπεράσματα:

Για τα συγκεκριμένα epochs δεν φαίνεται κάποια σημαντική διαφορά. Το μοντέλο είναι ελαφρώς πιο αδύναμο. Αναμενόμενη συμπεριφορά. Σημειώνουμε πως τα πειράματα αυτά είναι ενδεικτικά.

Γενικό Συμπέρασμα:

Η ορθή εκπαίδευση ενός μοντέλου είναι πολυπαραγωντική διαδικασία. Απαιτεί εναρμόνιση μεταξύ δεκάδων παραμέτρων. Τα αποτελέσματα δύναται να μην συμφωνούν με τις θεωρητικές προβλέψεις εάν δεν έχουμε εξασφαλίσει ,για παράδειγμα, επαρκή αριθμό training epochs ή ένα δεν έχουμε τροποποιήσει κατάλληλα την υπόλοιπη αρχιτεκτονική. Είναι διαδικασία που συνδυάζει θεωρητική γνώση, αλλά και trial and error. Η καλή γνώση της γενικότερης συμπεριφοράς είναι υψίστης σημασίας. Οι δοκιμές απαιτούν πολύ χρόνο και δεν είναι βιώσιμο να γίνονται μάταια. Μέθοδοι όπως Grid Search για fine tuning παραμέτρων είναι ανήκουστες χωρίς πολύ ικανό υπολογιστή (με GPU) ή πρόσβαση σε cloud computing. Τα δύο τελευταία πειράματα σχεδιάστηκαν ώστε να είναι ενδεικτικά. Δεν έδωσαν πολύ βαρυσήμαντα αποτελέσματα. Δεν επιτρέψαμε στα δίκτυα να συγκλίνουν όσο θα έπρεπε. Παρατηρώντας τις καμπύλες όμως, βλέπουμε πως η επίδοση έχει αρχίσει να σταθεροποιείται. Δεν απέχουν πολύ από την τελική τους μορφή. Δεν είναι μάταια, αφού παίρνουμε μια γενική ιδέα για την ταχύτητα σύγκλισης και κυρίως τις διαφορές σε χρόνο εκτέλεσης.

5.0. Σύγκριση με k-NN και NCC.

Υπάρχει σημαντική διαφορά στην επίδοση των Νευρωνικών Δικτύων και των απλούστερων αλγορίθμων ταξινόμησης της ενδιάμεσης εργασίας. Θυμόμαστε πως η μέγιστη απόδοση που είχε καταγραφεί στην ενδιάμεση εργασία ήταν της τάξης του 40% (από 1-NN με PCA). Ένα απλό CNN εύκολα φτάνει την διπλάσια επίδοση. Έχουν πολύ μεγαλύτερο learning capacity. Είναι λογικό εάν αναλογιστούμε τον αριθμών των παραμέτρων που εμπλέκονται στην αρχιτεκτονική τους. Στην περίπτωση του NCC είχαμε δέκα παραμέτρους. Για ένα απλό CNN μιλάμε για χιλιάδες. Η αυξημένη επίδοση, ωστόσο, συνοδεύεται από ραγδαία αύξηση του χρόνου εκπαίδευσης. Στην πρώτη περίπτωση μετράμε δευτερόλεπτα, ενώ στην δεύτερη ώρες για ένα μεσαίας κλίμακας δίκτυο. Οι εν λόγω διαπιστώσεις αφορούν image classification tasks. Σε πολύ απλούστερες περιπτώσεις ταξινόμησης δεν αποκλείεται οι απλοί αλγόριθμοι να αποδίδουν επαρκώς καλά με δεδομένο την πολύ χαμηλή χρονική περιπλοκότητα.

6.0. Παραδοτέο - Προσωπικό Σημείωμα.

Εκτός από το μοντέλο Kasper, είχε γραφτεί κώδικας για άλλα δύο έτοιμα μοντέλα. Ένα ResNet 18 και ένα MobileNet V2. Ήθελα να κάνω επιτροπή και να συγκρίνω επιδόσεις. Συνειδητοποίησα μετά το πόσο υποτίμησα τον χρόνο εκπαίδευσης. Εγκαταλείφθηκαν μετά από κάποια πειραματικά runs και δεν περιλαμβάνονται πλέον στο .ipynb. Θεώρησα δεδομένο πως για την αξιολόγηση της δεν θα γίνουν τα training runs. Οπότε για όποια αλλαγή στην αρχιτεκτονική του μοντέλου ένα overwrite τα cells στο Jupyter. Φαίνεται η τελευταία δοκιμή που έγινε. Γι' αυτό επισυνάπτω και φάκελο με τα βάρη και performance dictionaries και φάκελο με ready-to-copy-and-paste .txt αρχεία με τα configurations των πειραμάτων.